CIS 3300 · OVERVIEW

Exceptions, Assertions, and Logging

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Text:** *Core Java, Volume I* (13th ed.), Chapter 7
- **Materials:** four parts of class notes, with every example tested
- **Prerequisite you already have:** file I/O with text, CSV, and JSON files; classes, records, and interfaces; and inheritance and polymorphism, including constructor chaining and overriding

1. The short version

Until now, our programs have assumed that everything goes right. This unit is about what happens when it does not. Java reports a problem by throwing an *exception*, an object that travels up the chain of method calls until some code catches it. You will learn the difference between *checked* exceptions, which the compiler makes you plan for, and *unchecked* ones, which usually mean a bug, and how to catch, clean up, and keep a program running when one piece of data is bad.

Exceptions are classes, so everything from the inheritance unit applies: they form a hierarchy, a `catch` block is polymorphic, you can write your own exception classes with constructor chaining, and overriding methods must follow rules about what they may throw.

The unit finishes with two related tools. *Assertions* check assumptions in your own code while you develop it. *Logging* replaces debugging print statements with messages that have levels, can be turned up or down without editing code, and can go to a file. The last topic is logging safely, including what really happened with the Log4Shell vulnerability.

2. What We Will Cover

- **Throwing and catching.** Where errors come from, `throw`, reading a stack trace, the `Throwable` hierarchy, checked and unchecked exceptions, `throws` clauses, `try` and `catch`, the order of `catch` blocks, multi-catch, `finally`, and try-with-resources, including suppressed exceptions.
- **Designing with exceptions.** Choosing standard exceptions, writing your own exception classes, deciding between checked and unchecked, translating and chaining exceptions, keeping a program running past bad data, exceptions in overriding methods and constructors, and good habits.
- **Assertions.** The `assert` statement, why assertions are off by default, why they are worth using anyway, and when an exception is the better choice.
- **Logging.** `java.util.logging` with levels, handlers, and formats, `System.Logger`, the logging libraries used in industry, and logging securely: log injection, keeping secrets out of logs, and the lessons of Log4Shell.

3. Objectives

By the end of this unit, you should be able to:

1. Explain why exceptions are a better way to report errors than special return values.
2. Explain why exceptions are meant for exceptional circumstances, and avoid using them for decisions or loop control.
3. Throw an exception with a useful message, and read a stack trace to find where a problem started.
4. Place an exception class in the `Throwable` hierarchy, and classify it as checked or unchecked.
5. Apply the "catch or declare" rule for checked exceptions, using a `throws` clause when a method should pass an exception to its caller.
6. Write `try` and `catch` blocks that handle specific exceptions in the correct order, and use multi-catch where two exceptions need the same handling.
7. Use `finally` and try-with-resources to guarantee cleanup, and explain what a suppressed exception is.
8. Choose an appropriate standard exception for a given problem.
9. Write a custom exception class with constructors that chain to the superclass, and justify making it checked or unchecked.
10. Translate a low-level exception into one that fits your program, keeping the original as the cause.
11. Decide where to catch an exception so that one bad piece of data does not stop a program.
12. Decide, for each exception a program catches, whether it should recover and continue or end gracefully.
13. Explain the rules for exceptions in overriding methods and constructors, and connect them to the substitution principle.
14. Use assertions to check internal assumptions, enable them for testing, and explain when an exception should be used instead.
15. Use `java.util.logging` to record messages at appropriate levels, send them to the console and a file, and change the output format.
16. Describe log injection and the Log4Shell vulnerability, and apply safe logging practices.

4. How to Show Up Ready

- **Read ahead.** Skim Core Java, Volume I, Chapter 7, starting with how Java classifies exceptions.
- **Look back at your file I/O code.** Find every `throws IOException` and every try-with-resources statement you have written. This unit explains what they were doing.
- **Review constructor chaining and overriding.** Both come back, because exception classes are ordinary classes in a hierarchy.

- **Bring a working Eclipse setup.** We will cause errors on purpose and read the results. Know where to find **Run Configurations**, because you will add VM arguments to turn on assertions and change the logging format.
- **Run the examples yourself.** Type them in, break them, and predict the output before you run them. Reading a stack trace is a skill that only comes from practice.